Protecting the ALU Logo on the Ethereum Blockchain

Assignment 1 – Cryptocurrency and Digital Assets · Student: S. Munyankindi

Abstract

This project registers the official ALU logo on a local Ethereum blockchain as an ERC-721 NFT that stores the file's SHA-256 fingerprint, its registrant and a timestamp, so anyone can check if a logo file is genuine. A second ERC-20 contract splits ownership of the logo into 1,000,000 ALUT share tokens. It solves the problem of proving authenticity and ownership of a digital asset without trusting a single database holder.

Introduction

A logo is easy to copy, edit and falsely claim. ALU needs a way for anyone – a student, partner or employer – to confirm that a file is the official version. A normal database is controlled by one administrator who can edit or delete records silently, and a copyright registration proves a claim to a legal office but cannot be checked instantly by a stranger, says nothing about whether a given file was altered, and is slow and jurisdiction-specific. A blockchain record is public, timestamped, tamper-evident and verifiable by anyone in seconds with no permission. Registering the file's hash proves *this exact file existed and was claimed by this wallet at this time*, and any modified fake yields a different hash.

Blockchain Setup

I installed Node.js, then created an npm project and installed Hardhat 2.29.1, the Hardhat Toolbox (ethers, chai matchers) and OpenZeppelin Contracts 5.1.0. `hardhat.config.js` selects Solidity 0.8.28. Hardhat compiles the contracts, runs the tests on an in-memory chain, and `npx hardhat node` starts a local Ethereum node at `http://127.0.0.1:8545` with 20 funded test accounts of 10,000 fake ETH each.

Local blockchain: a private Ethereum network on your own computer. Developers use it because it is free (no real ETH), instant (blocks mined on demand), resettable, and mistakes cannot lose real money or become permanent on a public chain.

Wallet address: a 42-character hex identifier (e.g. `0xf39F…2266`) derived from a public key. It represents an account that can hold ETH, NFTs and tokens and sign transactions with its private key.

[SCREENSHOT: output of `npx hardhat node` showing the test wallet addresses]

Smart Contract Explanation

Generating the hash. I downloaded the logo from alueducation.com (`alu-logo.png`, 500×188) and ran `shasum -a 256 alu-logo.png`. The result is: `727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560`. The deploy script and tests recompute the same value with Node's `crypto` module.

[SCREENSHOT: terminal showing the SHA-256 hash of alu-logo.png]

ALUAssetRegistry (ERC-721). It defines `AssetMetadata` (name, file type, content hash as bytes32, registrant, timestamp), a mapping from token ID to metadata and a mapping from hash to bool.

- `registerAsset(name, fileType, contentHash)` – reverts with "Asset already registered: duplicate content hash" if the hash is known; otherwise increments the counter, stores the metadata, marks the hash used, mints the NFT to the caller with `_safeMint`, emits `AssetRegistered` and returns the token ID. State is written before minting to avoid re-entrancy problems.
- `verifyLogoIntegrity(tokenId, contentHash)` – a view function comparing the supplied hash to the stored one; returns `(true, "Logo is authentic.")` or `(false, "Warning: logo does not match.")`. It costs no gas when called off-chain.
- `getAsset(tokenId)` – a view function returning the whole struct.

What is a smart contract? Code stored and executed on the blockchain by every node, whose rules and state nobody can secretly change. A normal server program runs on a machine one company controls, which can alter code or data at will; a smart contract's execution is public, deterministic and enforced by the network.

NFT and ERC-721. A non-fungible token is a unique on-chain token with its own ID. ERC-721 is right for a logo because the logo is one unique, indivisible asset: each token ID is distinct and has a single owner and attached metadata, unlike interchangeable ERC-20 coins.

SHA-256. A one-way function that turns any file into a fixed 256-bit fingerprint. Because of the avalanche effect, flipping one pixel changes the input bytes, and the output changes completely and unpredictably (about half the bits flip), so edits cannot hide and the original cannot be reconstructed from the hash.

Immutability. Data is stored in blocks linked by hashes and replicated on thousands of nodes. Changing an old record would alter that block's hash and break every later block, and would need to be accepted by the majority of the network; the contract also has no function that edits or deletes a record.

ALULogoToken (ERC-20). Name "ALU Logo Token", symbol ALUT, fixed supply of 1,000,000 minted in the constructor to the owner address. `distributeShares(recipient, amount)` is `onlyOwner`, requires `amount > 0` and transfers tokens from the owner. `ownershipPercentage(account)` returns `balance × 100 / totalSupply` as a whole number (500,000 tokens → 50). `deploy.js` deploys the registry, registers the logo, then deploys the token to the deployer.

Tokenisation

Tokenisation is representing ownership of an asset as digital tokens on a blockchain so it can be split, tracked and transferred. For the ALU logo, the president's office starts with all 1,000,000 ALUT and could give shares to

faculty who help develop the brand, the student council, the administration/brand team and partner institutions or alumni leaders, rewarding contribution and giving stakeholders a formal stake. ERC-721 is for one unique item (the logo registration); ERC-20 is for many identical, divisible units (shares), which is why I used ERC-721 for registration and ERC-20 for ownership.

If a faculty member holds 100,000 of 1,000,000 ALUT, they own 10% of the logo's tokenised ownership. In practice that is a provable, transferable 10% stake, and could carry voting or revenue-share rights if ALU defined them (the token alone does not grant legal rights; those would need a legal framework).

Test Results

[SCREENSHOT: `npx hardhat test` showing 8 passing]

Result of my run: 8 passing – registration returns token ID 1; duplicate hash rejected; verify true for correct hash; verify false for wrong hash; getAsset returns the right name/type; full supply minted; distributeShares transfers correctly; ownershipPercentage returns 50 for 500,000 tokens.

Conclusion

I learned how hashing gives a tamper-evident fingerprint, how ERC-721 and ERC-20 serve different purposes, and how OpenZeppelin 5 and Hardhat fit together in testing. With more time I would deploy to the Sepolia testnet, store the logo on IPFS and add `tokenURI`, restrict registration to ALU-approved wallets, add events and a front-end (the next assignment's dApp).

GitHub Link

[INSERT PUBLIC GITHUB REPOSITORY URL]

Video

[INSERT VIDEO LINK]